Práctico 08 — Texturas y modelado del cielo

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

16 de octubre de 2026



Como cambiar la apariencia del terreno

Antecedentes: terreno usando malla indexada, relieve desde el ruido, recorte de caras traseras, pintado de un color plano.

Requisitos del proyecto

- Terreno
- Cielo

Como completar requisitos

- Poder decidir el color **punto por punto** de una superficie, y no vértice por vértice.
- Eso es una **textura**.

Hasta ahora el color es un valor por vértice que el rasterizador interpola. Hoy pasa a ser una **función sobre la superficie**: se guarda aparte, en una imagen, y se puede cambiar por fragmento.

Una textura es más que una imagen pegada

Una textura es un **arreglo de datos guardado en la memoria de la GPU**, que se indexa con coordenadas normalizadas y del que se obtiene un valor **por fragmento** aplicando interpolación.

Tres elementos que definen una textura:

El tipo

- **1D** — un degradé, una rampa de color.
- **2D** — el caso clásico: una imagen.
- **3D** — un volumen.
- **Cubemap** — seis arreglos 2D; se indexa con una **dirección**.

El formato

- Cómo está representada la información de color
- Cuántos canales y con cuántos bits por canal: `GL_RGBA8`, `GL_RGB8`, `GL_R8`...
- Fija el tamaño en memoria y no se puede cambiar después.

Las reglas de muestreo

- Que hacer cuando:
- 1. La textura es más grande: **filtrado**.
- 2. La textura es más chica: **repetición**.

Dos definiciones que no hay que mezclar

El elemento de una textura se denomina **texel**. No es un píxel: un píxel es del framebuffer y un texel es la información. La correspondencia **rara vez** es uno a uno (un texel corresponde a un pixel).

Las coordenadas de textura 2D (caso típico)

Cada vértice lleva un par (u, v) con valores en $[0, 1]$ que indica **qué punto de la textura hay que usar para sombrear**. Luego, la etapa de digitalización (rasterizado) los **interpola** entre los tres vértices del triángulo, exactamente igual que interpola el color.

Es un atributo más del vértice

```
struct Vertex
{
    glm::vec3 position;
    glm::vec3 normal;
    glm::vec2 tex_coords;  // <-- hoy
};
```

Coordenadas de textura e imagen

- En OpenGL, $(0, 0)$ es la esquina de **abajo a la izquierda**.
- En un archivo de imagen, la primera fila es la de **arriba**.
- Son convenciones opuestas.

El error clásico al cargar una textura

Al leer y cargar sin convertir la textura sale dada vuelta verticalmente. Hay dos soluciones: 1. invertir la imagen al cargarla o 2. invertir la coord. v en el shader. **Atención:** aplicar uno solo: sino se invierte otra vez.

¿Y si una coordenada (u, v) está fuera de $[0, 1]$?

¿De dónde salen las coordenadas (u, v)?

El terreno: salen al construir la grilla

```
tex_coords.x = float(j) / N;
tex_coords.y = float(i) / N;
```

Con (i, j) recorriendo las N celdas quedan en $[0, 1]$: **una sola tesela estirada sobre todo el terreno**. El factor de repetición se aplica después.

El cilindro: proyección cilíndrica

```
tex_coords.x = float(j) / segments;
tex_coords.y = float(i) / subdivisions;
```

El ángulo da la u y la altura la v.

El cono: proyección de disco

```
tex_coords.x = (cos(lon) + 1.0f) / 2.0f;
tex_coords.y = (sin(lon) + 1.0f) / 2.0f;
```

Otra convención: la textura se aplica como vista desde arriba, con el ápice en el centro. No está mal — es distinto el resultado que se obtiene—

La unión (costura)

El cilindro recorre $j = 0 \ldots N$, o sea un gajo de más. Ese vértice repetido está puesto **a propósito**: la misma posición necesita $u = 0$ de un lado y $u = 1$ del otro.

Todo esto es para geometría **generada por código**. Un modelo importado (`.obj`, `.gltf`) **ya trae sus** (u, v) en el archivo.

Un fragmento casi nunca pide un texel

Entre la textura y la imagen final hay una proyección en el medio por lo que **la correspondencia nunca es uno a uno**. Hay dos problemas distintos:

Ampliación (*magnification*)

- La textura es **chica** para el área que cubre.
- Un texel se reparte entre **muchos** fragmentos.
- Ejemplo: objeto muy cerca: cartel en primer plano.
- **Cómo se ve:** en **bloques** con `GL_NEAREST`, **borroso** con `GL_LINEAR`.

Reducción (*minification*)

- La textura es **grande** para el área que cubre.
- Un fragmento abarca **muchos** texels y hay que reducirlos en un solo color.
- Ejemplo: objetos distantes: el terreno hacia el horizonte.
- **Cómo se ve:** a continuación

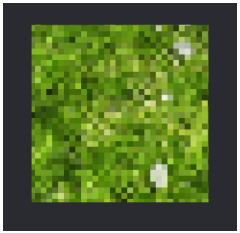
En OpenGL hay dos formas de manejar esto

`GL_TEXTURE_MAG_FILTER` y `GL_TEXTURE_MIN_FILTER`. Se fijan por separado, admiten valores distintos, y el segundo tiene **seis** opciones contra dos del primero — porque el problema de la reducción es más difícil.

Filtrado: qué hacer cuando no hay un texel justo

`GL_NEAREST`

Se toma el texel **más cercano**. Rápido, y respeta el dato exactamente como está.



`GL_LINEAR`

Se promedian los **cuatro** texels vecinos, pesados por distancia. Es el valor por omisión razonable.

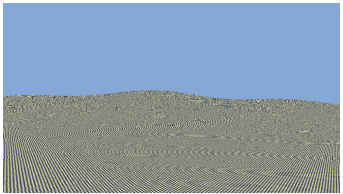


La misma textura de 256×256 ampliado unas veinte veces. A la izquierda se ven los texels; a la derecha se ven mezclados.

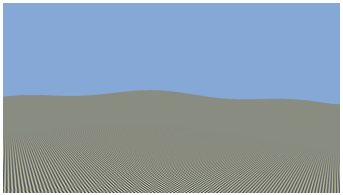
`GL_NEAREST` tiene función específica: es lo que se quiere cuando el dato **no es una imagen** — arte de píxel, un mapa de índices, una tabla de valores donde promediar dos celdas no significa nada.

Reducción: promediar no soluciona el problema

GL_LINEAR — sin mipmaps



GL_LINEAR_MIPMAP_LINEAR



Hacia el horizonte un fragmento abarca **decenas de texels**, y promediar los cuatro vecinos no los representa: el resultado depende de los cuatro texels en donde quedó el fragmento.

Solución: la pirámide de mipmaps

Se guarda la textura **varias veces, reduciendo a la mitad cada vez**, luego el hardware elige el nivel según cuánto se achicó. Para 1024×1024 son **11 niveles**.

El costo de memoria es $+33{,}33\,\%$ ($1/4 + 1/16 + \ldots = 1/3$): ej. 4,000 MiB pasan a **5,333 MiB**.

Repetición: qué pasa fuera de $[0, 1]$

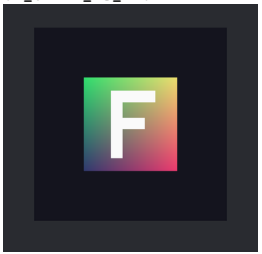
GL_REPEAT



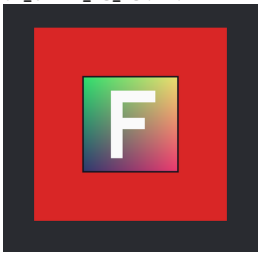
GL_MIRRORED_REPEAT



GL_CLAMP_TO_EDGE



GL_CLAMP_TO_BORDER



La misma textura muestreada con coordenadas de $-0{,}5$ a $1{,}5$

El caso del terreno

- La grilla tiene (u, v) en $[0, 1]$; el *shader* multiplica por 40 y `GL_REPEAT` hace el resto: la textura se repite 40 veces sobre 2 km.
- Cada repetición cubre 50 m. Sin repetición, la textura abarcaría 2 km.

El caso del fondo

- `GL_CLAMP_TO_EDGE` en las tres direcciones.
- Las direcciones se llaman `S`, `T` y `R` — no x, y, z.
- Con `GL_REPEAT` el filtrado toma texels del otro extremo y **se ven las costuras** de las aristas.

Primero hay que leer la imagen

En OpenGL no hay funcionalidades para manejo de archivos. Sólo procesa bytes, por lo que primero hay que decodificar una imagen que este comprimida con algún formato (PNG,JPG,etc.)

Una herramienta para esto es `stb_image`: una librería de sólo encabezados orientada al procesamiento sencillo de imagenes y algunas otras funcionalidades para Computación Gráfica.

```
#define STB_IMAGE_IMPLEMENTATION      // Mandatorio: una sola vez en todo el proyecto
#include <stb/stb_image.h>

stbi_set_flip_vertically_on_load(1);  // invertir para el origen de OpenGL abajo
int w, h, canales;
unsigned char* datos = stbi_load("imagen.png", &w, &h, &canales, 4);
// ... se carga a la GPU ...
stbi_image_free(datos);               // liberar la copia del CPU
```

El último argumento **fuerza** la cantidad de canales: usando 4 siempre se decodifica como RGBA, independiente del contenido del archivo. En `canales` se lee los canales del archivo, no como se procesa.

Crear una textura en OpenGL

Cuatro etapas

1. **Crear** un objeto. Devuelve un identificador, igual que un buffer o un VAO.
2. **Reservar** el almacenamiento: tamaño, formato y cuántos niveles de mipmap. No se puede cambiar después.
3. **Llenar** con los datos de la imagen, nivel por nivel — o llenar el nivel 0 y pedir que se generen los demás.
4. **Parametrizar** el muestreo: los dos filtros y los modos de repetición del bloque anterior.

Lo importante de la etapa 2

El almacenamiento es **inmutable**: una vez reservado, no se puede cambiar ni el tamaño ni el formato ni la cantidad de niveles. Para otro tamaño hay que crear una textura nueva.

Es una ventaja, no una limitación. El driver sabe desde el principio cuánta memoria hace falta y con qué disposición, y puede verificar de una sola vez que la textura está completa.

Las cuatro etapas usando DSA

```
GLuint tex;
glCreateTextures(GL_TEXTURE_2D, 1, &tex);                          // 1. crear
glTextureStorage2D(tex, niveles, GL_RGBA8, w, h);                  // 2. reservar
glTextureSubImage2D(tex, 0, 0, 0, w, h,                            // 3. llenar
                    GL_RGBA, GL_UNSIGNED_BYTE, datos);
glGenerateTextureMipmap(tex);                                      //    y la piramide
glTextureParameteri(tex, GL_TEXTURE_MIN_FILTER,                    // 4. parametrizar
                    GL_LINEAR_MIPMAP_LINEAR);
glTextureParameteri(tex, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTextureParameteri(tex, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTextureParameteri(tex, GL_TEXTURE_WRAP_T, GL_REPEAT);
```

Dos formatos: `GL_RGBA8` en 2 es cómo se guarda **en el GPU**; `GL_RGBA` + `GL_UNSIGNED_BYTE` en 3 es el formato usado para la decodificación de la imagen (datos). Pueden no coincidir y OpenGL se encarga de convertir.

Usar las texturas: unidades de textura

Un *shader* puede usar varias texturas a la vez, así que la GPU tiene **unidades de textura** numeradas. Para pintar con una textura hay que **asignar la textura a una unidad** (del lado del CPU) y en el *shader* elegir esa unidad para leer.

```
glBindTextureUnit(0, tex);   // unidad 0
glUseProgram(programa);
glBindVertexArray(vao);
glDrawElements(...);
```

```
layout (binding = 0)
uniform sampler2D uTextura;

void main() {
  fragColor = texture(uTextura, fUV);
}
```

El `uniform uTextura` es el **número de unidad** (0, 1, 2...): no es el identificador de la textura!

Con `layout (binding = 0)` en el *shader* no hace falta programar el `uniform` del lado del CPU. Sino hay que usar `glProgramUniform1i(programa, loc, 0)`.

Modelar el cielo: dos métodos clásicos

Skybox — usando un cubo

- Seis imágenes, una por cara.
- Se muestrea con una **dirección**.
- Admite cualquier cielo: nubes, montañas en el horizonte, una ciudad.

Skydome — usando una semiesfera

- Una textura, o directamente un **degradé 1D** indexado por la elevación.
- Sin aristas y sin costuras.
- Barato de generar: un cielo despejado son 64 texels.

Característica común de los dos modelos

Generan un superficie que **rodea al observador** y que se dibuja **sin profundidad**: no está cerca pero tampoco está lejos, se encuentra *detrás de todo*. El objeto geométrico es un detalle; la ilusión es la misma.

¿Cómo se logra que un cubo parezca un cielo?

La ilusión: el cielo no tiene tamaño

Hay que lograr que el cubo **siempre esté centrado en el ojo**: el observador se mueve y el cielo no cambia. Sin paralaje, el sentido de visión no tiene como determinar la distancia y lo percibe como **infinitamente lejano**.

Solución:

1. Eliminar de la matriz de vista la **traslación** y dejar sólo la rotación:

```
glm::mat4 vista_sin_traslacion = glm::mat4(glm::mat3(vista));
```

2. Resolver la **profundidad** porque el cielo tiene que quedar detrás de todo. Dos métodos que no se mezclan:

(a) Dibujar primero, sin escribir profundidad

```
glDepthMask(GL_FALSE);
// ... dibujar el cielo ...
glDepthMask(GL_TRUE);
// ... dibujar la escena ...
```

(b) Dibujar al último, con la profundidad al máximo

```
glDepthFunc(GL_LEQUAL);
// en el vertex shader del cielo:
gl_Position = pos.xyww;  // z = w
```

El cubo tiene que estar dentro del volumen de visualización:

Con un cubo de lado 2 y el plano cercano en 0.5, el plano cercano corta al cubo y el cielo sale como una franja.

Textura cubemap: se indexa con una dirección

Una textura en formato cubemap no se muestrea con (u, v): se muestrea con un **vector de tres componentes**. El hardware decide qué cara cruza ese rayo y dónde la cruza. La dirección es, simplemente, **la posición del vértice del cubo**: desde el centro, cada vértice *es* una dirección.

```
in vec3 fDireccion;
layout (binding = 0)
uniform samplerCube uCielo;
...
fragColor = texture(uCielo, fDireccion)
    ;
```

El orden de las caras es fijo: $+X$ derecha, $-X$ izquierda, $+Y$ arriba, $-Y$ abajo, $+Z$ frente, $-Z$ atrás.

Dos detalles a tener en cuenta: las caras **no** se invierten al cargarlas (al revés que una textura 2D) y la repetición es `GL_CLAMP_TO_EDGE` en `S`, `T` y `R`.

```
glCreateTextures(GL_TEXTURE_CUBE_MAP, 1, &t);
glTextureStorage2D(t, 1, GL_RGB8, w, h);                          // se reserva en 2D
for (int cara = 0; cara < 6; ++cara)
    glTextureSubImage3D(t, 0, 0,0, cara, w,h,1, GL_RGB, ...);    // y se llena en 3D
```

El costo del cubemap: las seis caras **son** seis capas. Usando un tamaño de 1024 × 1024 en `GL_RGB8`, son **18 MiB** — unas **diez veces** el terreno entero: malla 1,8 MiB más textura 0,25 MiB.

El problema de la altitud

El cielo sigue al observador mientras que el terreno no. En el ejemplo el borde del terreno termina a 1 km.

A **100 m** la banda es angosta: 5,4°



A **600 m** una "alfombra verde flotando"



La misma escena con el mismo encuadre: lo único que cambia es la altitud. El horizonte del fondo queda siempre a la altura del ojo (el cubo está centrado en el ojo), el límite del terreno "se hunde" y entre los dos aparece una banda que es la **parte del cubemap por debajo del horizonte**.

altitud	borde del terreno	la banda mide
100 m	5,71°	5,4°
600 m	30,96°	30,2°
1000 m	45,00°	44,0°

A 1000 m de altitiud el terreno completo sale del cuadro (terreno de 200 celdas de 10 m, 2 km de lado).

¿Se arregla agrandando el terreno?

El problema de la altitud: cómo solucionarlo

Cómo es la realidad física

- El horizonte real *sí* baja con la altitud — la Tierra es redonda.
- Pero baja **muy poco**: 0,32° a 100 m, 0,72° a 500 m, 1,76° a 3000 m.
- Frente a la banda medida es **entre 18 y 44 veces más chico**: la curvatura no es la causa.

El problema del terreno *finito*

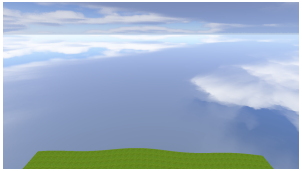
- Para que el borde se vea sobre el horizonte real, la malla tiene que **llegar** al horizonte.
- A h = 1000 m la ventana de terreno tendría que medir **más de 100 km** de lado.
- Con el paso de 10 m son **más de 3000 veces** las celdas y la memoria.
- No se llega al horizonte con la malla uniforme.

La causa es que el terreno es **finito**: termina antes del horizonte.

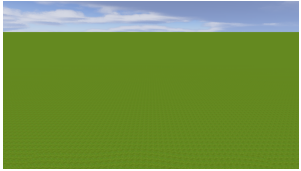
¿Cómo se disimula sin agrandar la malla?

El problema de la altitud: continuar el suelo en el fondo

A **1000 m**, sólo el cubemap: la banda



La misma escena, con el **suelo continuado**



- Cada píxel del fondo es una **dirección** $\vec{d}$. Si está por debajo del horizonte ($d_y < 0$), el rayo $\vec{P}(t) = \vec{O} + t\,\vec{d}$ corta el plano del suelo $y = y_s$ en $t = (O_y - y_s)/(-d_y)$.
- En ese punto se lee la **textura del terreno**, con $(u, v) = (x, z)$ dividido por los metros de una tesela: la misma escala que la malla
- Hacia el horizonte $t \to \infty$ y el **mipmap** promedia solo: de lejos queda el color medio del pasto.

Lo que no resuelve

- El plano continúa la textura, **no la forma**: con montañas en el borde vuelve a hacer falta una ventana de terreno grande. Los simuladores extienden el terreno de verdad, con niveles de detalle (LOD).
- Queda un horizonte con un salto de color. Se soluciona con **niebla** en la lejanía: Unidad IX.

El trabajo de la semana

Práctico 08 — lo que hay que hacer

- **Texturar el terreno** del Práctico 07 para darle una apariencia de suelo.
- **Mirar las coordenadas de las primitivas** del Práctico 04 con una textura aplicada para verificar que están bien.
- **El cielo con un cubemap**: seis caras, la traslación fuera de la vista y una de las dos recetas de profundidad.
- **Implementar alguna solución** para la "banda" de terreno que falta al subir en altitud (puede ser continuando el suelo en el fondo).

Lo que queda abierto

- Hoy la textura reemplaza el color. En la clase de iluminación se va a multiplicar por la luz difusa, y ahí la normal deja de ser una caja negra.
- La niebla termina de fundir el terreno con el horizonte para mejorar la apariencia de la escena (Unidad IX).
- La textura 1D (como idea del skydome) está como alternativa en el enunciado para experimentar con otra forma de pintar el cielo.
- No olvidarse del checkpoint de proyecto el **27-nov**; el cielo debería formar parte de la demo.